

Verificação, Validação e Teste de *Software*

Prof. Dr. Leonardo Souza Silva

Módulo 4 — Ferramentas de Teste de *Software*

Unidade 2 - Ferramentas e Automação de Testes

Neste ponto da disciplina...

- Entendemos o que é Qualidade de *Software*;
- Compreendemos como as atividades de Verificação, Validação e Teste de *Software* podem apoiar o **processo de desenvolvimento e manutenção**.
 - Objetivo é construir um *software* com qualidade.

Ferramentas e Automação de Teste

- O processo de Teste de *Software* pode ser apoiado pelo uso de ferramentas:
 - Apoio ao Testador na execução do plano de testes;
 - Implementação de alguns tipos de teste e critérios;
 - Suporte a diferentes linguagens, e
 - Automação do processo de teste.

Ferramentas de Teste de Software

Ferramentas de teste de software	Teste de unidade	Teste de integração	Teste de sistema	Critérios funcionais	Critérios baseados em fluxo de controle	Critérios baseados em fluxo de dados	Critérios baseados em mutação	Linguagem suportada	Exigência de código-fonte	Atividade de depuração	Teste de regressão
<i>JProbe Developer Suite</i>	✓		✓		✓			Java	✓	✓	
<i>Parasoft C++ Test</i>	✓		✓	✓	✓			C/C++	✓		✓
<i>Parasoft Insure++</i>								C/C++	✓	✓	
<i>Purify Plus</i>			✓					C/C++	✓	✓	
<i>TestComplete</i>			✓	✓				Várias linguagens		✓	✓
<i>Sikuli</i>			✓	✓				Várias linguagens			
<i>JUnit</i>	✓			✓				Java			
<i>Cobertura</i>	✓				✓			Java			
<i>Emma</i>	✓				✓			Java			
<i>CodeCover</i>	✓				✓			Java			✓
<i>Parasoft JTest</i>	✓		✓	✓	✓			Java	✓		✓
<i>JaBUTi</i>	✓				✓	✓		Java		✓	✓
<i>BA-DUA</i>	✓				✓	✓		Java		✓	
<i>TCAT/Java</i>	✓				✓			Java	✓		
<i>JCover</i>	✓				✓			Java	✓		
<i>ProLint</i>								C/C++	✓		
<i>Rational PureCoverage</i>	✓		✓		✓			C++/Java	✓		
<i>MuJava</i>	✓	✓					✓	Java	✓		
<i>PIT</i>	✓						✓	Java	✓		

Fonte: Delamaro, 2016

Ferramentas de Teste de Software

Ferramentas de teste de software	Teste de unidade	Teste de integração	Teste de sistema	Critérios funcionais	Critérios baseados em fluxo de controle	Critérios baseados em fluxo de dados	Critérios baseados em mutação	Linguagem suportada	Exigência de código-fonte	Atividade de depuração	Teste de regressão
<i>JUnit</i>	✓			✓				Java			
<i>Cobertura</i>	✓				✓			Java			
<i>Emma</i>	✓				✓			Java			
<i>CodeCover</i>	✓				✓			Java			✓
<i>Parasoft JTest</i>	✓		✓	✓	✓			Java	✓		✓
<i>JaBUTi</i>	✓				✓	✓		Java		✓	✓

Fonte: Delamaro, 2016 (Editado)

Ferramentas de Teste de Software

- JUnit
 - *Framework* de teste popular, permite documentar e executar casos de teste de forma automática;
 - Relata quais casos de teste não se comportaram conforme a especificação;
 - Não é possível acompanhar a cobertura do código pelos casos de teste;
 - Pode ser usado mesmo se o *bytecode* e a especificação do programa estejam disponíveis.

Ferramentas de Teste de Software

- JaBUTi (*Java Bytecode Understanding and Testing*)
 - Apoia a aplicação de critérios de teste estruturais;
 - Não necessita do código-fonte → *bytecode*;
 - Pode ser com qualquer linguagem que gere um código objetivo compatível com a especificação da JVM.

E na prática?

calculaMedia.py (Módulo 1 - Unidade 2)

- Para demonstrar o funcionamento de uma ferramenta, vamos usar o programa calculaMedia.py e a ferramenta *Pytest*.

```
calculaMedia.py X test_calculaMedia.py  
app > calculaMedia.py > calculaMedia > aritmetica  
1 class calculaMedia:  
2  
3     def aritmetica(num1, num2):  
4         return (num1 + num2)/2  
5
```

Fonte: Os autores



Fonte: Pytest. 2025

- Ambiente de trabalho que permite escrever, de maneira muito fácil, testes para aplicações e bibliotecas em linguagem *Python*.
 - Permite a execução de teste de unidade (*Python*).
- Mais informações: <https://docs.pytest.org/en/stable/>

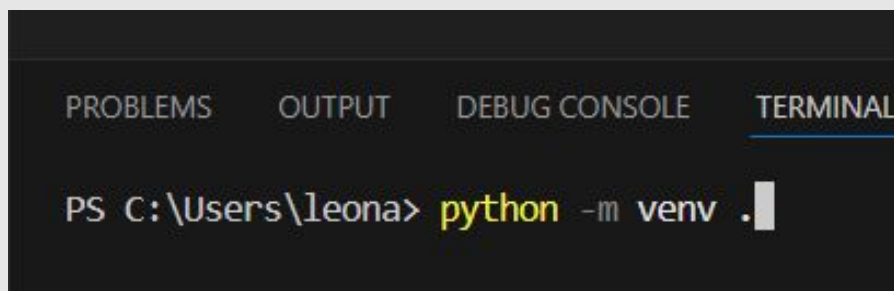
Passos necessários

- Configuração do ambiente;
- Instalação do *Pytest*;
- Construção do programa de teste com os casos de teste.
- Execução do teste.

A descrição detalhada dos passos seguidos para realização deste teste pode ser encontrada [neste tutorial](#).

Configuração do Ambiente 1/2

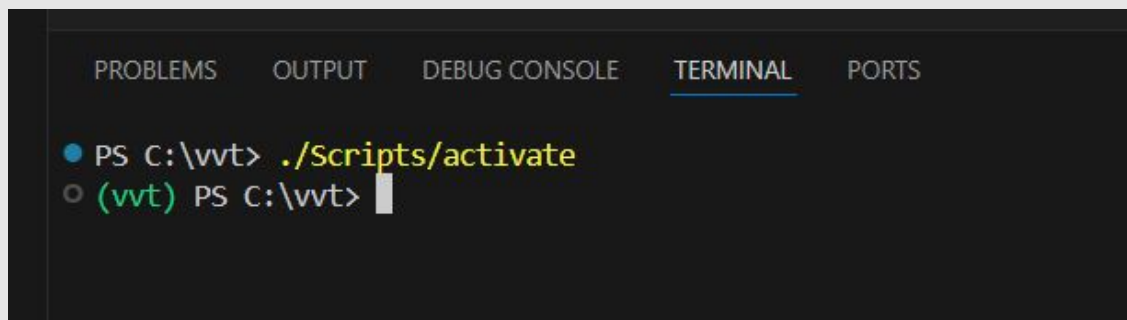
- Instalação



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
```

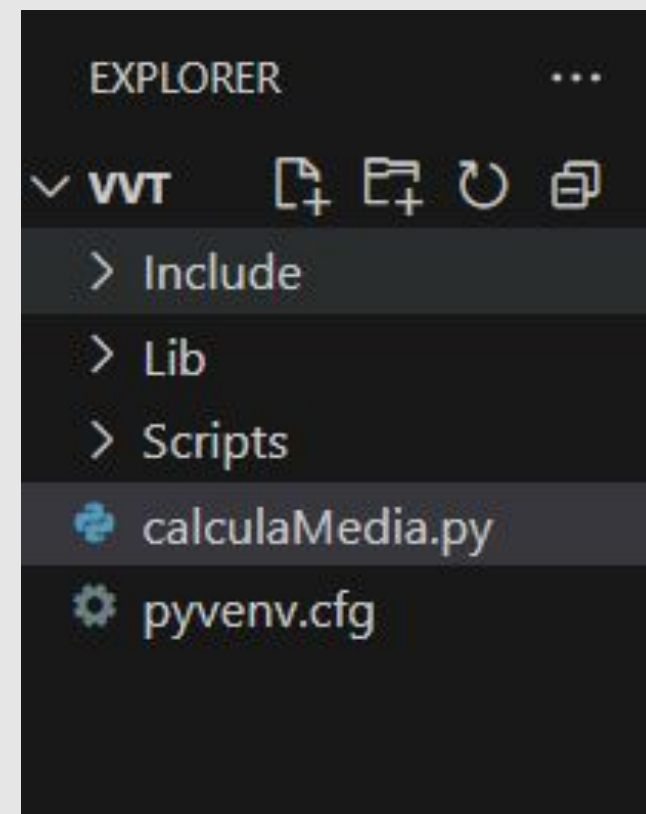
```
PS C:\Users\leona> python -m venv .
```

- Ativação



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
```

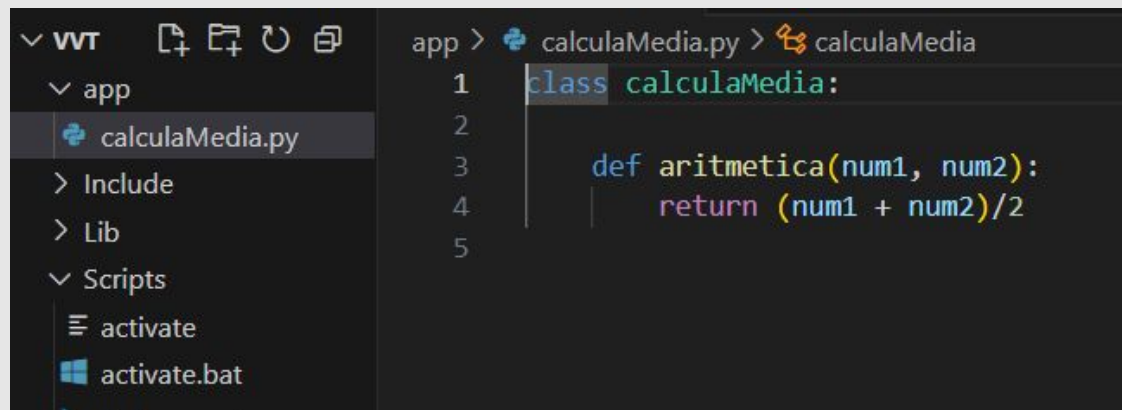
```
● PS C:\vvt> ./Scripts/activate  
○ (vvt) PS C:\vvt>
```



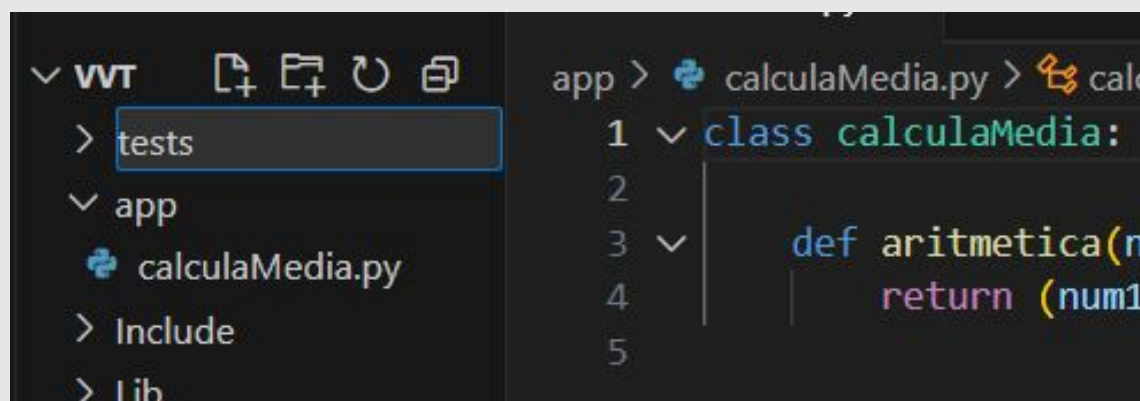
Fonte: Os autores

Configuração do Ambiente 2/2

- Criar os diretórios **app** e **tests**



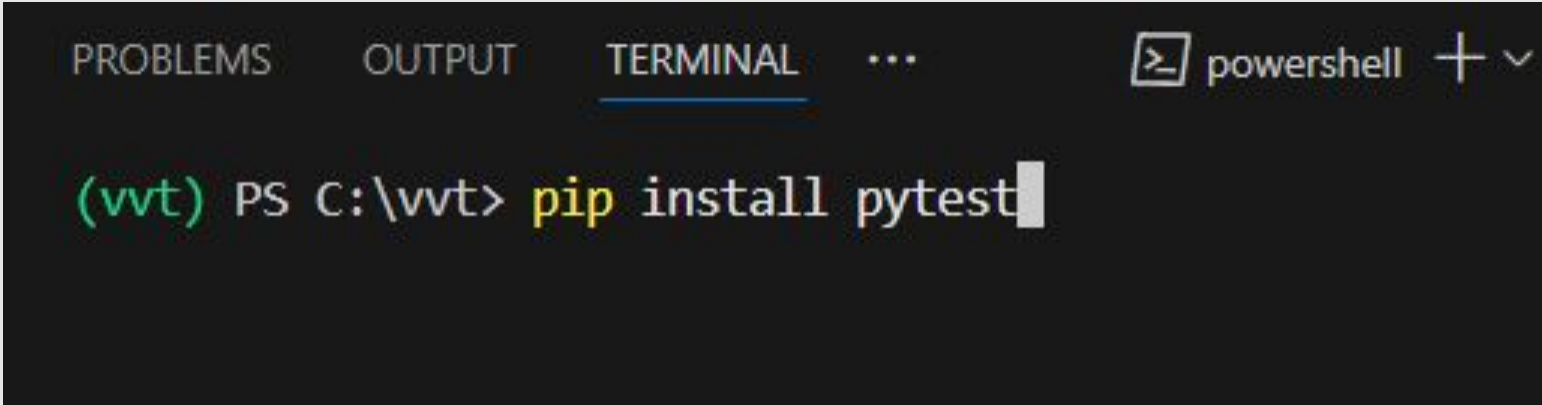
```
app > calculaMedia.py > calculaMedia
1 class calculaMedia:
2
3     def aritmetica(num1, num2):
4         return (num1 + num2)/2
5
```



```
app > calculaMedia.py > calculaMedia
1 class calculaMedia:
2
3     def aritmetica(n
4         return (num1
5
```

Fonte: Os autores

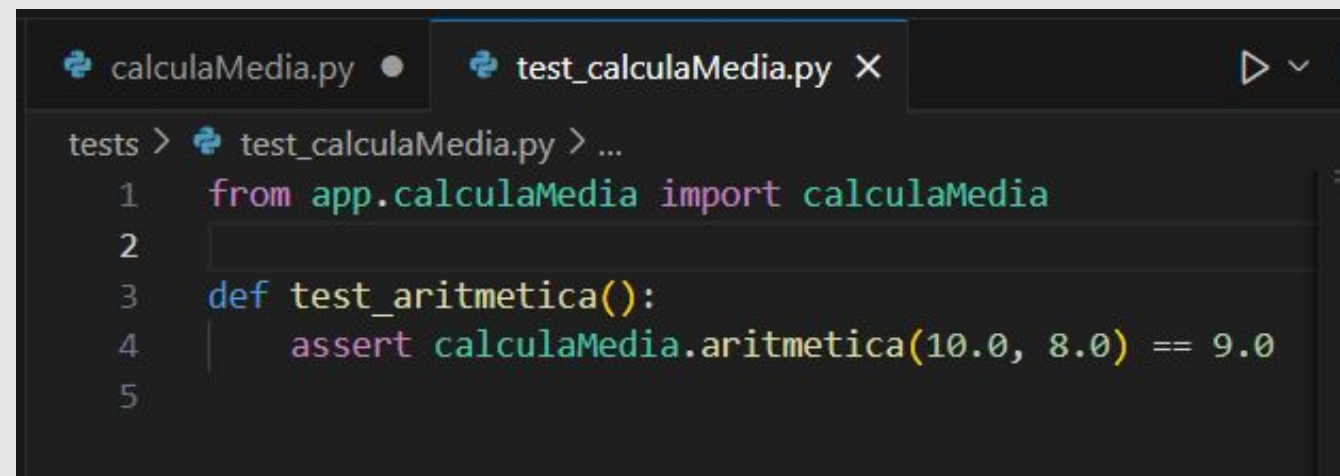
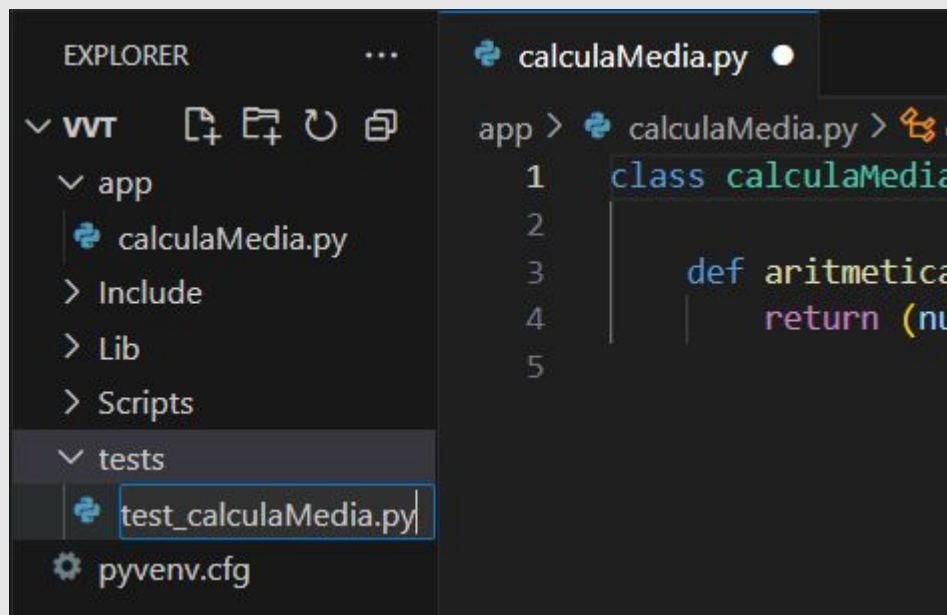
Instalação do



```
PROBLEMS OUTPUT TERMINAL ... powershell + v  
(vvt) PS C:\vvt> pip install pytest
```

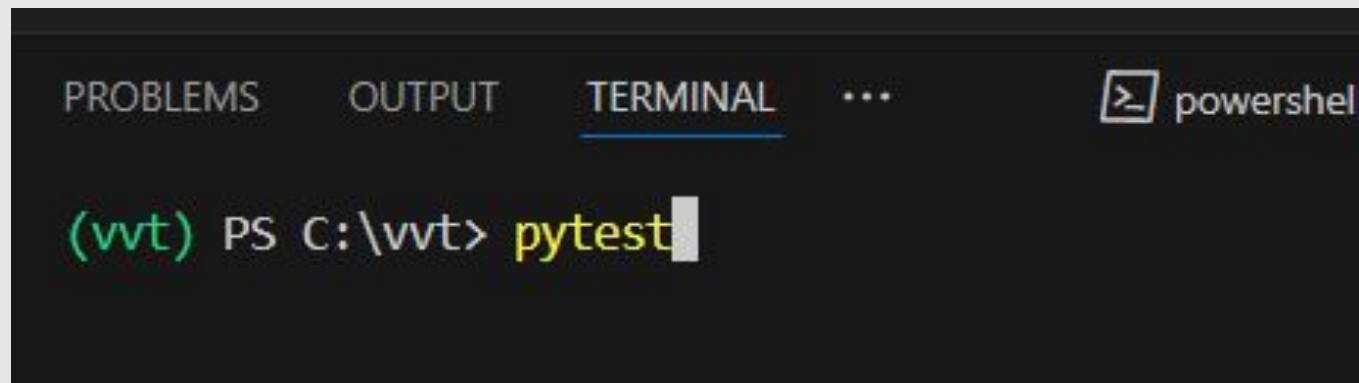
Fonte: Os autores

Criação dos Casos de Teste



Fonte: Os autores

Execução do pytest



```
PROBLEMS  OUTPUT  TERMINAL  ...  powershell

(vvt) PS C:\vvt> pytest
```

Fonte: Os autores

Execução do — Resultado



```

PROBLEMS OUTPUT TERMINAL ... powershell + ▢ ▢ ... ^ ×
(vvt) PS C:\vvt> pytest
===== test session starts =====
platform win32 -- Python 3.12.8, pytest-8.3.5, pluggy-1.5.0
rootdir: C:\vvt
collected 53 items

Lib\site-packages\colorama\tests\ansi_test.py ... [ 5%]
Lib\site-packages\colorama\tests\ansitowin32_test.py ..... [ 24%]
..... [ 49%]
Lib\site-packages\colorama\tests\initialise_test.py ssssssssss. [ 69%]
[ 69%]
Lib\site-packages\colorama\tests\isatty_test.py ..... [ 83%]
Lib\site-packages\colorama\tests\winterm_test.py ..... [ 98%]
tests\test_calculaMedia.py . [100%]

===== 43 passed, 10 skipped in 0.73s =====
(vvt) PS C:\vvt>

```

Fonte: Os autores

Execução do pytest

— Resultado Negativo

```
calculaMedia.py × test_calculaMedia.py
app > calculaMedia.py > calculaMedia > aritmetica
1 class calculaMedia:
2
3     def aritmetica(num1, num2):
4         return num1 + num2/2
5
```

Fonte: Os autores

```
(vvt) PS C:\vvt> pytest
===== test session starts =====
platform win32 -- Python 3.12.8, pytest-8.3.5, pluggy-1.5.0
rootdir: C:\vvt
collected 53 items

Lib\site-packages\colorama\tests\ansi_test.py ... [ 5%]
Lib\site-packages\colorama\tests\ansitowin32_test.py ..... [ 24%]
..... [ 49%]
Lib\site-packages\colorama\tests\initialise_test.py ssssssssss. [ 69%]
[ 69%]
Lib\site-packages\colorama\tests\isatty_test.py ..... [ 83%]
Lib\site-packages\colorama\tests\winterm_test.py ..... [ 98%]
tests\test_calculaMedia.py F [100%]

===== FAILURES =====
_____ test_aritmetica _____

    def test_aritmetica():
>     assert calculaMedia.aritmetica(10.0, 8.0) == 9.0
E       assert 14.0 == 9.0
E         + where 14.0 = <function calculaMedia.aritmetica at 0x000001B58BAE8E00>(10.0, 8.0)
E         + where <function calculaMedia.aritmetica at 0x000001B58BAE8E00> = calculaMedia.aritmetica

tests\test_calculaMedia.py:4: AssertionError
===== short test summary info =====
FAILED tests/test_calculaMedia.py::test_aritmetica - assert 14.0 == 9.0
===== 1 failed, 42 passed, 10 skipped in 0.70s =====
```

Execução do

```
calculaMedia.py  test_calculaMedia.py X  Search vvt — test_calculaMedia.py
tests > test_calculaMedia.py > test_aritmetica
1  from app.calculaMedia import calculaMedia
2
3  def test_aritmetica():
4      assert calculaMedia.aritmetica(10.0, 8.0) == 9.0
5      assert calculaMedia.aritmetica(5.5, 7.0) == 6.25
6      assert calculaMedia.aritmetica(7, 8.5) == 7.75
7
```

```
PROBLEMS  OUTPUT  TERMINAL  ...  powershell + v [ ] [ ] ... ^ X

collected 53 items

Lib\site-packages\colorama\tests\ansi_test.py ... [ 5%]
Lib\site-packages\colorama\tests\ansitowin32_test.py ..... [ 24%]
..... [ 49%]
Lib\site-packages\colorama\tests\initialise_test.py ssssssssss. [ 69%]
[ 69%]
Lib\site-packages\colorama\tests\isatty_test.py ..... [ 83%]
Lib\site-packages\colorama\tests\winterm_test.py ..... [ 98%]
tests\test_calculaMedia.py . [100%]

===== 43 passed, 10 skipped in 0.66s =====
(vvt) PS C:\vvt>
```

Fonte: Os autores

Exemplo: calculaMedia.py

- Objetivo de dar um primeiro contato com esse tipo de ferramenta;
- Outras ferramentas surgindo e em uso pelos desenvolvedores de *software*;
 - Cobertura de Código:
 - Coverage, Codecov, dentre outras.
- Integração com repositórios, por exemplo, GitHub.

Para finalizar...

- Entendemos o uso de ferramentas e da automação na atividade de teste de *software*.
- Compreendemos sua importância para o processo de teste e manutenção de *software*.

Referências

BENNETT, Keith H.; RAJLICH, Václav T. **Software maintenance and evolution: a roadmap**. In: Proceedings of the Conference on the Future of Software Engineering. 2000. p. 73-87.

DELAMARO, Márcio Eduardo. **Introdução ao teste de software**. 2. Rio de Janeiro: GEN LTC, 2016. 1 recurso online. (SBC (Sociedade Brasileira de Computação)). ISBN 9788595155732. **Capítulo 1. Disponível na Biblioteca Digital da UFMS.**

Pytest - helps you write better programs. Disponível em: <https://docs.pytest.org/en/stable/index.html>. Acesso em: 25 mar. 2025.

Licenciamento



Respeitadas as formas de citação formal de autores de acordo com as normas da ABNT NBR 6023 (2018), a não ser que esteja indicado de outra forma, todo material desta apresentação está licenciado sob uma [Licença Creative Commons - Atribuição 4.0 Internacional](https://creativecommons.org/licenses/by/4.0/).

